MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

NAME OF FACULTY: RAJESH KUMAR

SUB & CODE: Database Management Systems (4CS4-04, 4AD4-04, 4AM4-04)

SEMESTER: IV

SESSION: 2023-24 (Even SEM.)

BRANCH: CS & AI/DS

BATCH: A & B

UNIT: V

SUBMISSON: 15-3-2024

Lecture No.	Topics Covered	Page No.
1	Implementation of Cocurrency	1-2
2	Lock Based Protocols	3-4
3	Time Stamp Based Protocols and Validation Based Protocols	5-9
4	Deadlock Handling	10-14
5	Database Failure and Recovery	15-17
6	Recovery Schemes	18-21
7.	Recovery with Concurrent Transaction	22-25

References:

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F Korth
2. Fundamentals of Database Systems by Elmsari, Navathe, 5th Edition Pearson

Faculty HOD Dean Academics 16/3/24

UNIT-5 (Implementation of Concurrency)
LECTURE NO: -33

Implementation of Concurrency

What is concurrency control in DBMS?

Concurrency control concept comes under the Transaction in database management system (DBMS). It is a procedure in DBMS which helps us for the management of two simultaneous processes to execute without conflicts between each other, these conflicts occur in multi user systems.

Concurrency can simply be said to be executing multiple transactions at a time. It is required to increase time efficiency. If many transactions try to access the same data, then inconsistency arises. Concurrency control required to maintain consistency data.

Advantages

The advantages of concurrency control are as follows −

- Waiting time will be decreased.
- Response time will decrease.
- Resource utilization will increase.
- System performance & Efficiency is increased.

Control concurrency

The simultaneous execution of transactions over shared databases can create several data integrity and consistency problems.

For example, if too many people are logging in the ATM machines, serial updates and synchronization in the bank servers should happen whenever the transaction is done, if not it gives wrong information and wrong data in the database.

Main problems in using Concurrency

The problems which arise while using concurrency are as follows −

Updates will be lost −One transaction does some changes and another transaction deletes that change. One transaction nullifies the updates of another transaction.

Uncommitted Dependency or dirty read problem −On variable has updated in one transaction, at the same time another transaction has started and deleted the value of the variable there the variable is not getting updated or committed that has been done on the first transaction this gives us false values or the previous values of the variables this is a major problem.

MITRC

Inconsistent retrievals−One transaction is updating multiple different variables, another transaction is in a process to update those variables, and the problem occurs is inconsistency of the same variable in different instances.

Why use Concurrency method?

Reasons for using Concurrency control method is DBMS:

- To apply Isolation through mutual exclusion between conflicting transactions
- To resolve read-write and write-write conflict issues
- To preserve database consistency through constantly preserving execution obstructions
- The system needs to control the interaction among the concurrent transactions. This control is achieved using concurrent-control schemes.
- Concurrency control helps to ensure serializability.

Example

Assume that two people who go to electronic kiosks at the same time to buy a movie ticket for the same movie and the same show time.

However, there is only one seat left in for the movie show in that particular theatre. Without concurrency control in DBMS, it is possible that both moviegoers will end up purchasing a ticket. However, concurrency control method does not allow this to happen. Both moviegoers can still access information written in the movie seating database. But concurrency control only provides a ticket to the buyer who has completed the transaction process first.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-5 (Lock-based protocols)
LECTURE NO: -34

Lock-based protocols

Concurrency Control Protocols

Different concurrency control protocols offer different benefits between the amount of concurrency they allow and the amount of overhead that they impose. Following are the Concurrency Control techniques in DBMS:

- Lock-Based Protocols
- Two Phase Locking Protocol
- Timestamp-Based Protocols
- Validation-Based Protocols

Lock-based Protocols

Lock Based Protocols in DBMS is a mechanism in which a transaction cannot Read or Write the data until it acquires an appropriate lock. Lock based protocols help to eliminate the concurrency problem in DBMS for simultaneous transactions by locking or isolating a particular transaction to a single user.

A lock is a data variable which is associated with a data item. This lock signifies that operations that can be performed on the data item. Locks in DBMS help synchronize access to the database items by concurrent transactions.

Types of Locks in DBMS

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

Shared Lock (lock-S)

Exclusive Lock (lock-X)

1.Shared Lock-Shared Locks are basically the locks that grant Read-Only access to the data items associated with it. This means when a shared lock is implemented on a database, it can be READ by multiple users, however, none of the users reading the data items will be able to update it. In other words, there is no write access associated with Shared Lock. Since multiple users can read the data items simultaneously, more than one shared lock can be placed on them at a given time but for the shared lock to be placed, the data item must not have any other locks associated with it.

MITRC

2. Exclusive lock-While Shared Lock allows Read-Only access, Exclusive Locks allow both Read and Write access on a data item present in the database. Often represented as lock-X(), Exclusive locks provide the privilege of reading and modifying the data as seen fit by a user. For the sake of security and consistency of the database, this process will be exclusive to the user using them and is not permitted to other users trying to access the same data item. Once the exclusive lock is placed, no other locks including the Shared lock can be placed on the same row or column until its unlocked.

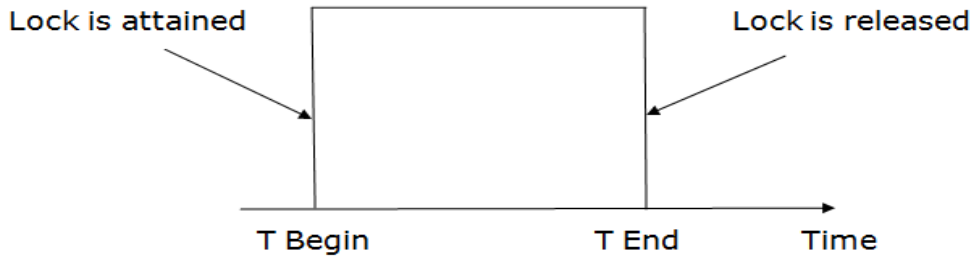
There are four types of lock protocols available:

1. Simplistic lock protocol

It is the simplest way of locking the data while transaction. Simplistic lock-based protocols allow all the transactions to get the lock on the data before insert or delete or update on it. It will unlock the data item after completing the transaction.

2. Pre-claiming Lock Protocol

- Pre-claiming Lock Protocols evaluate the transaction to list all the data items on which they need locks.
- Before initiating an execution of the transaction, it requests DBMS for all the lock on all those data items.
- If all the locks are granted then this protocol allows the transaction to begin. When the transaction is completed then it releases all the lock.
- If all the locks are not granted then this protocol allows the transaction to rolls back and waits until all the locks are granted.



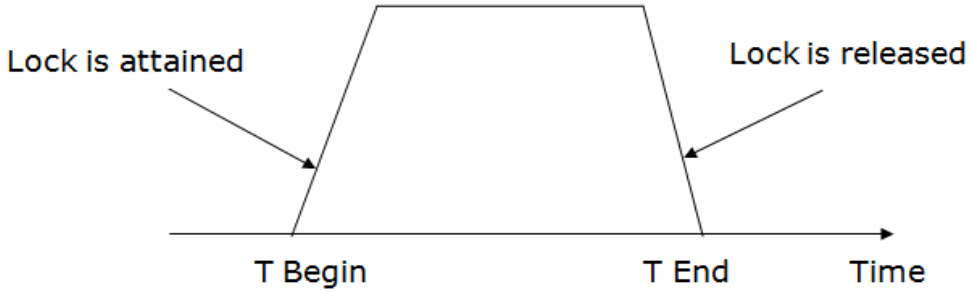
3.Two-phase locking (2PL)

The two-phase locking protocol divides the execution phase of the transaction into three parts.

In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.

In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock.

In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.



There are two phases of 2PL:

Growing phase: In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

Shrinking phase: In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-5 (Timestamp-based protocols and Validation-based protocols)

LECTURE NO: -35

Timestamp Ordering Protocol

The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.

The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.

The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.

Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.

The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Advantages:

- Schedules are serializable just like 2PL protocols
- No waiting for the transaction, which eliminates the possibility of deadlocks.

Disadvantages:

Starvation is possible if the same transaction is restarted and continually aborted.

Validation Based Protocol

Validation phase is also known as optimistic concurrency control technique. In the validation-based protocol, the transaction is executed in the following three phases:

Read phase: In this phase, the transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.

Validation phase: In this phase, the temporary variable value will be validated against the actual data to see if it violates the serializability.

Write phase: If the validation of the transaction is validated, then the temporary results are written to the database or system otherwise the transaction is rolled back.

Timestamp-based protocols and Validation-based protocols

Here each phase has the following different timestamps:

Start (Ti): It contains the time when Ti started its execution.

Validation (Ti): It contains the time when Ti finishes its read phase and starts its validation phase.

Finish (Ti): It contains the time when Ti finishes its write phase.

- This protocol is used to determine the time stamp for the transaction for serialization using the time stamp of the validation phase, as it is the actual phase which determines if the transaction will commit or rollback.
- Hence TS(T) = validation(T).
- The serializability is determined during the validation process. It can't be decided in advance.
- While executing the transaction, it ensures a greater degree of concurrency and also les Validation-based protocols s number of conflicts.
- Thus, it contains transactions which have less number of rollbacks.

Characteristics of Good Concurrency Protocol

An ideal concurrency control DBMS mechanism has the following objectives:

- Must be resilient to site and communication failures.
- It allows the parallel execution of transactions to achieve maximum concurrency.
- Its storage mechanisms and computational methods should be modest to minimize overhead.
- It must enforce some constraints on the structure of atomic actions of transactions.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-5 (Deadlock handling)
LECTURE NO: -36

Deadlock in DBMS

A deadlock is a condition where two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most troubled complications in DBMS as no task ever gets finished and is in waiting state forever.

For example: In the student table, transaction T1 holds a lock on some rows and needs to update some rows in the grade table. Simultaneously, transaction T2 holds locks on some rows in the grade table and needs to update the rows in the Student table held by Transaction T1.

Now, the main problem arises. Now Transaction T1 is waiting for T2 to release its lock and similarly, transaction T2 is waiting for T1 to release its lock. All activities come to a halt state and remain at a standstill. It will remain in a standstill until the DBMS detects the deadlock and aborts one of the transactions.

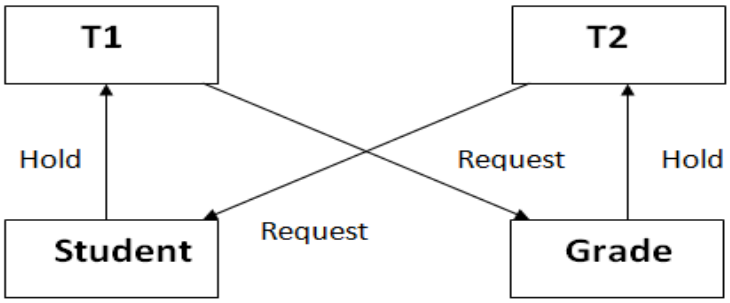


Figure: Deadlock in DBMS

Deadlock Handling in Centralized Systems

There are three classical approaches for deadlock handling, namely –

- Deadlock prevention.
- Deadlock avoidance.
- Deadlock detection and removal.

Deadlock Prevention

The deadlock prevention approach does not allow any transaction to acquire locks that will lead to deadlocks. The convention is that when more than one transaction request for locking the same data item, only one of them is granted the lock.

Deadlock Avoidance

The deadlock avoidance approach handles deadlocks before they occur. It analyses the transactions and the locks to determine whether or not waiting leads to a deadlock.

Deadlock Detection and Removal

The deadlock detection and removal approach run a deadlock detection algorithm periodically and removes deadlock in case there is one. It does not check for deadlock when a transaction places a request for a lock. When a transaction requests a lock, the lock manager checks whether it is available. If it is available, the transaction is allowed to lock the data item; otherwise the transaction is allowed to wait.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-5 (Database Failure and Recovery)
LECTURE NO: -37

Database Failure: -

To find that where the problem has occurred, we generalize a failure into the following categories:

1. Transaction failure
2. System crash
3. Disk failure

1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be -

Logical errors: If a transaction cannot complete due to some code error or an internal error condition, then the logical error occurs.

Syntax error: It occurs where the DBMS itself terminates an active transaction because the database system is not able to execute it. For example, the system aborts an active transaction, in case of deadlock or resource unavailability.

2. System Crash

System failure can occur due to power failure or other hardware or software failure. Example: Operating system error.

Fail-stop assumption: In the system crash, non-volatile storage is assumed not to be corrupted.

3. Disk Failure

It occurs where hard-disk drives or storage drives used to fail frequently. It was a common problem in the early days of technology evolution.

Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.

What is Data Recovery?

It is the method of restoring the database to its correct state in the event of a failure at the time of the transaction or after the end of a process. Earlier, you have been given the concept of database recovery as a service that should be provided by all the DBMS for ensuring that the database is dependable and remains in a consistent state in the

MITRC

presence of failures. In this context, dependability refers to both the flexibility of the DBMS to various kinds of failure and its ability to recover from those failures.

What is the need of Recovery of data?

The storage of data usually includes four types of media with an increasing amount of reliability: the main memory, the magnetic disk, the magnetic tape, and the optical disk. Many different forms of failure can affect database processing and/or transaction, and each of them has to be dealt with differently. Some data failures can affect the main memory only, while others involve non-volatile or secondary storage also. Among the sources of failure are:

- Due to hardware or software errors, the system crashes, which ultimately resulting in loss of main memory.
- Failures of media, such as head crashes or unreadable media that results in the loss of portions of secondary storage.
- There can be application software errors, such as logical errors that are accessing the database that can cause one or more transactions to abort or fail.
- Natural physical disasters can also occur, such as fires, floods, earthquakes, or power failures.
- Carelessness or unintentional destruction of data or directories by operators or users.

Recovery Facilities

Every DBMS should offer the following facilities to help out with the recovery mechanism:

- Backup mechanism makes backup copies at a specific interval for the database.
- Logging facilities keep tracing the current state of transactions and any changes made to the database.
- Checkpoint facility allows updates to the database for getting the latest patches to be made permanent and keep secure from vulnerability.
- Recovery manager allows the database system for restoring the database to a reliable and steady-state after any failure occurs.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

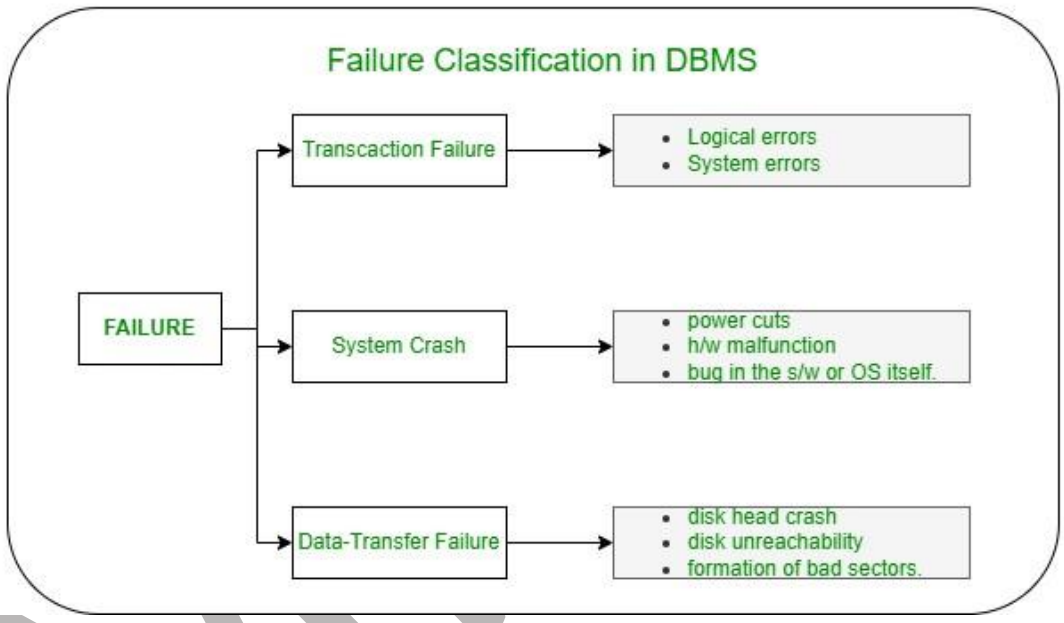
MITRC

UNIT-5 (Database Failure)
LECTURE NO: -38

Failure in terms of a database can be defined as its inability to execute the specified transaction or loss of data from the database. A DBMS is vulnerable to several kinds of failures and each of these failures needs to be managed differently. There are many reasons that can cause database failures such as network failure, system crash, natural disasters, carelessness, software errors, etc.

Failure Classification in DBMS

A failure in DBMS can be classified as:



1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be -

Logical errors: If a transaction cannot complete due to some code error or an internal error condition, then the logical error occurs.

Syntax error: It occurs where the DBMS itself terminates an active transaction because the database system is not able to execute it. For example, the system aborts an active transaction, in case of deadlock or resource unavailability.

2. System Crash

System failure can occur due to power failure or other hardware or software failure. Example: Operating system error.

Fail-stop assumption: In the system crash, non-volatile storage is assumed not to be corrupted.

3. Disk Failure

It occurs where hard-disk drives or storage drives used to fail frequently. It was a common problem in the early days of technology evolution.

Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-5 (Recovery Schemes)
LECTURE NO: -39

Database Recovery:

The Database are prone to failures due to inconsistency, network failure, errors or any kind of accidental damage. So, database recovery techniques are highly important to bring a database back into a working state after a failure. There are four different recovery techniques are available in the Database.

1. Mirroring

2. Recovery using Backups

3. Recovery using Transaction Logs

Mirroring:

Two complete copies of the database maintain on-line on different stable storage devices. This method mostly uses in environments that require non-stop, fault-tolerant operations.

Recovery using Backups:

Backups are useful if there has been extensive damage to database. Backups are mainly two types:

- Immediate Backup
- Archival Backup

Immediate Backup are kept in a floppy disk, hard disk or magnetic tapes. These come in handy when a technical fault occurs in the primary database such as system failure, disk crashes, network failure. Damage due to virus attacks repair using the immediate backup.

Archival Backup:

Archival Backups are kept in mass storage devices such as magnetic tape, CD-ROMs, Internet Servers etc. They are very useful for recovering data after a disaster such as fire, earthquake, flood etc. Archival Backup should be kept at a different site other than where the system is functioning. Archival Backup at a separate place remains safe from thefts and international destruction by user staff.

Recovery using Transaction Logs:

1. Shadow Paging
2. Log-Based Recovery

MITRC

1. **Shadow Paging-** The idea is to maintain two-page tables during the life of a transaction: the current page table and the shadow page table. When the transaction starts, both tables are identical. The shadow page is never changed during the life of the transaction. The current page is updated with each write operation. Each table entry points to a page on the disk. When the transaction is committed, the shadow page entry becomes a copy of the current page table entry and the disk block with the old data is released.

   If the shadow is stored in non-volatile memory and a system crash occurs, then the shadow page table is copied to the current page table. This guarantees that the shadow page table will point to the database pages corresponding to the state of the database prior to any transaction that was active at the time of the crash, making aborts automatic.

   Advantages

   The advantages of shadow paging are as follows −

   - No need for log records.
   - No undo/ Redo algorithm.
   - Recovery is faster.

   Disadvantages

   The disadvantages of shadow paging are as follows −

   - Data is fragmented or scattered.
   - Garbage collection problem. Database pages containing old versions of modified data need to be garbage collected after every transaction.
   - Concurrent transactions are difficult to execute.

Log-Based Recovery

The most widely used structure for recording database modifications is the log. The log is a sequence of log records, recording all the update activities in the database.

There are several types of log records. An update log record describes a single database write. It has these fields:

• Transaction identifier is the unique identifier of the transaction that performed the write operation.

• Data-item identifier is the unique identifier of the data item written. Typically, it is the location on disk of the data item.

MITRC

• Old value is the value of the data item prior to the write.

• New value is the value that the data item will have after the write.

There are many types of log record we denote the various types of log records:

1. <Ti start> transaction Ti has started.
2. <Ti, Xi, V1, V2> transaction Ti has performed a write on data item Xi, Xj has value v1 before the write and will have value V2 after the write operation.
3. <Ti commit> transaction Ti has committed.
4. <Ti abort> transaction Ti ha aborted.

Whenever a transaction performs write operation database is modified only when log record for that write is created. And logs may or may not contain the record of reading of data item. It is so because reading the data item does not affect the consistency of the database and is nowhere helpful in recovery mechanism.

In the recovery method, we use two operations:

Undo (Ti): Restores the values of all data item updated by transaction Ti to the old values.

Redo (Ti): Sets the value of all data item updated by transaction Ti to the new values. We need to undo a transaction T only when log contains the record <T start> only when log contains the record <start> and <T commit> both.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
3. C.J. Date: Data Base Design, Addison Wesley.

MITRC

UNIT-5 (Recovery with Concurrent transactions)

LECTURE NO: - 40

Recovery with Concurrent Transactions

Concurrency control means that multiple transactions can be executed at the same time and then the interleaved logs occur. But there may be changes in transaction results so maintain the order of execution of those transactions.

During recovery, it would be very difficult for the recovery system to backtrack all the logs and then start recovering.

Recovery with concurrent transactions can be done in the following four ways.

- Interaction with concurrency control
- Transaction rollback
- Checkpoints
- Restart recovery

Interaction with concurrency control:

In this scheme, the recovery scheme depends greatly on the concurrency control scheme that is used. So, to rollback a failed transaction, we must undo the updates performed by the transaction.

Transaction rollback:

In this scheme, we rollback a failed transaction by using the log.

The system scans the log backward a failed transaction, for every log record found in the log the system restores the data item.

Checkpoints:

- Checkpoints is a process of saving a snapshot of the applications state so that it can restart from that point in case of failure.
- Checkpoint is a point of time at which a record is written onto the database form the buffers.
- Checkpoint shortens the recovery process.
- When it reaches the checkpoint, then the transaction will be updated into the database, and till that point, the entire log file will be removed from the file. Then the log file is updated with the new step of transaction till the next checkpoint and so on.
- The checkpoint is used to declare the point before which the DBMS was in the consistent state, and all the transactions were committed.

MITRC

Recovery with Concurrent transactions

- In this scheme, we used checkpoints to reduce the number of log records that the system must scan when it recovers from a crash.
- In a concurrent transaction processing system, we require that the checkpoint log record be of the form <checkpoint L>, where **'L'** is a list of transactions active at the time of the checkpoint.
- A fuzzy checkpoint is a checkpoint where transactions are allowed to perform updates even while buffer blocks are being written out.

Restart recovery:

- When the system recovers from a crash, it constructs two lists.
- The undo-list consists of transactions to be undone, and the redo-list consists of transaction to be redone.
- The system constructs the two lists as follows: Initially, they are both empty. The system scans the log backward, examining each record, until it finds the first <checkpoint> record.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
3. C.J. Date: Data Base Design, Addison Wesley.

MITRC